Examen parcial - Parte Practica

Gerencia de Riesgos Financieros - Instrumento: Micron Technology (MU)

Setup

```
if (!require("pacman")) install.packages("pacman")
```

```
## Cargando paquete requerido: pacman
```

```
## Warning: package 'pacman' was built under R version 4.5.2
```

```
pacman::p_load(quantmod, tseries, aTSA, forecast, lmtest, ggplot2, FinTS)
options(scipen = 999)
```

Descarga de precios y calculo de retornos

```
# Datos descargados una sola vez y guardados en mu_datos.rds -- se leen de
# ahi (no de Yahoo en vivo) para que el resultado no cambie entre corridas.
mu_xts <- readRDS("mu_datos.rds")
precio <- Ad(mu_xts)
colnames(precio) <- "MU"

retorno <- na.omit(diff(log(precio)))

# Retornos oscilan alrededor de cero, sin tendencia -> ya estacionarios.
```

1. Estadisticas descriptivas de los retornos

```
media   <- mean(retorno) #EL retonro promedio diario es de 0,475%
desv    <- sd(retorno) #La volatilidad diaria es de 4,5% diario
minimo  <- min(retorno)  #La peor caida en un día fue de 17,65%
maximo  <- max(retorno)   #La mejor subida en un día fue de 17,64%
mediana <- median(retorno)   #La mediana es de 0,345%

c(media = media, sd = desv, min = minimo, max = maximo, mediana = mediana)
```

```
##        media           sd          min          max      mediana
##  0.004755733  0.045021605 -0.176486808  0.176400829  0.003458680
```

2. Raices unitarias sobre el precio

```
precio_num <- as.numeric(precio)

#Hacemos las pruebas de estacionariedad sobre la serie original:

#H0 = Hay raíz unitaria (no hay estacionariedad)
#H1 = No hay raíz unitaria (hay estacionariedad)
```

```
adf.test(precio_num)
```

```
## Augmented Dickey-Fuller Test
## alternative: stationary
##
## Type 1: no drift no trend
##      lag  ADF p.value
## [1,]   0 1.14   0.932
## [2,]   1 1.37   0.956
## [3,]   2 1.75   0.980
## [4,]   3 1.23   0.943
## [5,]   4 1.40   0.959
## [6,]   5 1.66   0.976
## Type 2: with drift no trend
##      lag   ADF p.value
## [1,]   0 0.105   0.964
## [2,]   1 0.290   0.977
## [3,]   2 0.586   0.989
## [4,]   3 0.179   0.970
## [5,]   4 0.318   0.978
## [6,]   5 0.523   0.986
## Type 3: with drift and trend
##      lag   ADF p.value
## [1,]   0 -1.76   0.678
## [2,]   1 -1.61   0.742
## [3,]   2 -1.40   0.832
## [4,]   3 -1.68   0.712
## [5,]   4 -1.57   0.760
## [6,]   5 -1.42   0.822
## ----
## Note: in fact, p.value = 0.01 means p.value <= 0.01
```

```
pp.test(precio_num)
```

```
## Phillips-Perron Unit Root Test
## alternative: stationary
##
## Type 1: no drift no trend
##  lag Z_rho p.value
##    5  1.75    0.98
## -----
##  Type 2: with drift no trend
##  lag Z_rho p.value
##    5 0.619    0.98
## -----
##  Type 3: with drift and trend
##  lag Z_rho p.value
##    5 -4.88   0.816
## ---------------
## Note: p-value = 0.01 means p.value <= 0.01
```

```
#Ya ADF y PP son suficientes, no rechazan H0, es decir hay raíz unitaria (no es
#no es estacionaria)
```

```r
#kpss.test(precio_num, lag.short = FALSE) / solo la aplicamos si adf y pp se
#contradicen

#Procedemos con la primera diferenciación, y realizamos de nuevo las pruebas:
precio_diff <- diff(precio_num)

adf.test(precio_diff)
```

```
## Augmented Dickey-Fuller Test
## alternative: stationary
##
## Type 1: no drift no trend
##      lag   ADF p.value
## [1,]   0 -24.4    0.01
## [2,]   1 -19.0    0.01
## [3,]   2 -11.8    0.01
## [4,]   3 -11.1    0.01
## [5,]   4 -10.9    0.01
## [6,]   5 -10.8    0.01
## Type 2: with drift no trend
##      lag   ADF p.value
## [1,]   0 -24.5    0.01
## [2,]   1 -19.1    0.01
## [3,]   2 -11.9    0.01
## [4,]   3 -11.3    0.01
## [5,]   4 -11.1    0.01
## [6,]   5 -11.0    0.01
## Type 3: with drift and trend
##      lag   ADF p.value
## [1,]   0 -24.5    0.01
## [2,]   1 -19.2    0.01
## [3,]   2 -12.0    0.01
## [4,]   3 -11.4    0.01
## [5,]   4 -11.2    0.01
## [6,]   5 -11.2    0.01
## ----
## Note: in fact, p.value = 0.01 means p.value <= 0.01
```

```r
pp.test(precio_diff)
```

```
## Phillips-Perron Unit Root Test
## alternative: stationary
##
## Type 1: no drift no trend
##  lag Z_rho p.value
##    5  -524    0.01
## -----
##  Type 2: with drift no trend
##  lag Z_rho p.value
##    5  -522    0.01
## -----
##  Type 3: with drift and trend
##  lag Z_rho p.value
##    5  -520    0.01
```

```
## ---------------
## Note: p-value = 0.01 means p.value <= 0.01
```

```
#Ambas coinciden y rechazan H0, ya la serie es estacionaria con d=1
```

3. Modelo ARIMA sobre los retornos

```
r <- as.numeric(retorno)
modelo <- auto.arima(r, stepwise = FALSE)
modelo
```

```
## Series: r
## ARIMA(2,0,2) with non-zero mean
##
## Coefficients:
##           ar1     ar2     ma1     ma2    mean
##       -1.8359  -0.935  1.8740  0.9560  0.0047
## s.e.   0.0355   0.041  0.0283  0.0299  0.0020
##
## sigma^2 = 0.001987:  log likelihood = 851.47
## AIC=-1690.93   AICc=-1690.76   BIC=-1665.62
```

```
coeftest(modelo)
```

```
##
## z test of coefficients:
##
##             Estimate Std. Error  z value             Pr(>|z|)
## ar1       -1.8358729  0.0355130 -51.6958 < 0.0000000000000002 ***
## ar2       -0.9350153  0.0409541 -22.8308 < 0.0000000000000002 ***
## ma1        1.8740243  0.0282775  66.2727 < 0.0000000000000002 ***
## ma2        0.9559827  0.0299457  31.9239 < 0.0000000000000002 ***
## intercept  0.0047454  0.0020109   2.3598              0.01828 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
#a. todos los coeficientes son significativos con un nivel de significancia del 10%
#   (de hecho al 0,1% salvo la constante, p=0,018). Ojo: esto es cierto pero
#   engañoso. El ARMA(2,2) esta sobreajustado: ar=(-1,836; -0,935) y ma=(1,874; 0,956),
#   cada coeficiente AR es casi la negacion del MA (modulos de raices 1,034 AR y 1,023 MA,
#   casi iguales y casi sobre la circunferencia unitaria), es decir las raices AR y MA
#   casi se cancelan entre si, y la "alta significancia" refleja ese par casi cancelado,
#   no una estructura predecible real. El criterio BIC prefiere un ARIMA(0,0,0)
#   (BIC -1676,97 vs -1665,62 del ARMA(2,2)) y el AIC solo mejora 5,5 puntos
#   (-1690,93 vs -1685,40 del modelo solo con constante). Los pronosticos oscilan
#   (-0,63%, +1,73%, -0,80%, +1,64%, -0,47%) por raices complejas cercanas a -1, no por
#   una dinamica predecible. La causa no es el stepwise: auto.arima con stepwise=TRUE
#   entrega el mismo ARIMA(2,0,2).
#b. Se necesitaron 2 rezagos de la parte autoregresiva, es decir retornos de sus propios
#   dos valores pasados y 2 rezagos dela parte de media movil, es decir, errores de los
#   ultimos 2 periodos.
```

4. Validacion de supuestos del modelo

```r
res <- residuals(modelo)

#Media cero (se cumple, p=0,9999) -- pero es trivial: los residuos de un modelo con
#intercepto tienen media ~0 por construccion, asi que no es evidencia de buen ajuste.
t.test(res)
```

```
##
##  One Sample t-test
##
## data:  res
## t = 0.0001879, df = 501, p-value = 0.9999
## alternative hypothesis: true mean is not equal to 0
## 95 percent confidence interval:
##  -0.003892404  0.003893149
## sample estimates:
##       mean of x
## 0.0000003722996
```

```r
#No autocorrelación (se cumple). El modelo estima 4 parametros ARMA (p+q=4), por lo que
#se corrige con fitdf=4 (gl=6): p=0,147 (sin la correccion daba 0,4848). La conclusion
#(no hay autocorrelacion residual) no cambia.
Box.test(res, lag = 10, type = "Ljung-Box", fitdf = 4)
```

```
##
##  Box-Ljung test
##
## data:  res
## X-squared = 9.5065, df = 6, p-value = 0.147
```

```r
#Homocedasticidad (no se cumple) -> Se aplica GARCH en escenarios de heterocedasticidad.
#La heterocedasticidad condicional esta bien detectada (Ljung-Box de res^2 p=3,5e-6;
#ARCH-LM p=0,0012). Aclaracion: este codigo NO ajusta un modelo ARCH/GARCH; solo
#diagnostica la necesidad. (Ajustarlo no se pidio en esta parte; no se agrega aqui.)
Box.test(res^2, lag = 10, type = "Ljung-Box")
```

```
##
##  Box-Ljung test
##
## data:  res^2
## X-squared = 43.873, df = 10, p-value = 0.000003469
```

```r
FinTS::ArchTest(res, lags = 10)
```

```
##
##  ARCH LM-test; Null hypothesis: no ARCH effects
##
## data:  res
## Chi-squared = 29.068, df = 10, p-value = 0.001215
```

```r
#normalidad (no se cumple; JB=78,8, p<2,2e-16). La no normalidad no sesga mucho los
#estimadores puntuales, pero SI invalida los intervalos de pronostico del 95%: forecast()
#los calcula bajo normalidad e ignora la heterocedasticidad condicional, por lo que
#esos intervalos no son confiables.
jarque.bera.test(res)
```

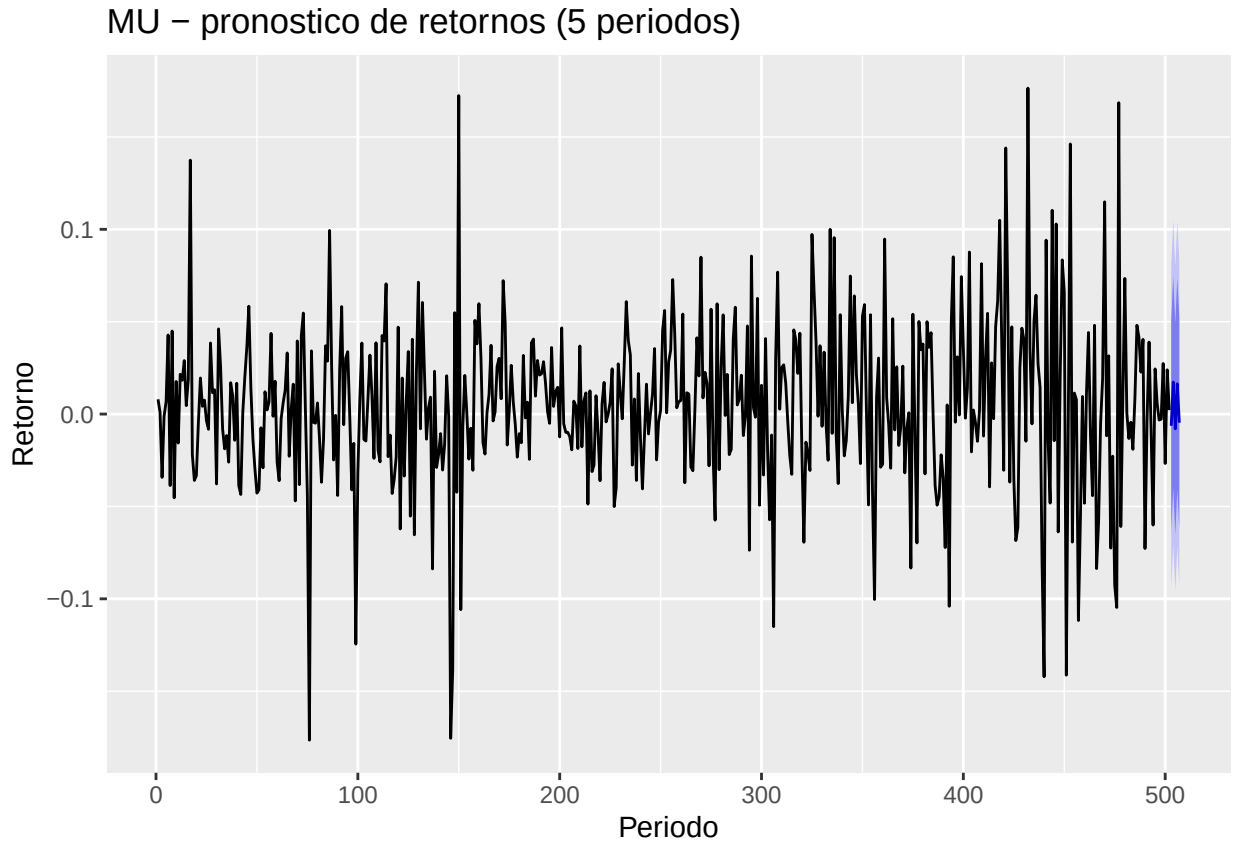
```
##
##  Jarque Bera Test
##
## data:  res
## X-squared = 78.757, df = 2, p-value < 0.00000000000000022
```

5. Pronostico de los siguientes 5 periodos

```
pron <- forecast(modelo, h = 5)
pron
```

```
##     Point Forecast       Lo 80      Hi 80       Lo 95      Hi 95
## 503   -0.006332694 -0.06345287 0.05078748 -0.09369047 0.08102508
## 504    0.017321939 -0.03983979 0.07448367 -0.07009939 0.10474327
## 505   -0.007985363 -0.06521578 0.04924506 -0.09551174 0.07954101
## 506    0.016358183 -0.04095660 0.07367296 -0.07129721 0.10401358
## 507   -0.004670758 -0.06206856 0.05272704 -0.09245311 0.08311160
```

```
autoplot(pron) +
  labs(title = "MU - pronostico de retornos (5 periodos)", x = "Periodo", y = "Retorno")
```




```
#INTERPRETACIÓN 1ER pronóstico: Se pronóstica que en el periodo 503 el retorno sea de -0,63%
#Con un nivel de confianza del 95% se estima que el retorno esté entre -9,37% y +8,10%
#(intervalo basado en normalidad, ver la advertencia de la seccion 4). Los 5 pronosticos
#oscilan de signo (-0,63%, +1,73%, -0,80%, +1,64%, -0,47%): es un artefacto del
#sobreajuste del ARMA(2,2) (raices casi canceladas), no una senal predecible.
```